Control de un robot mediante gestos de la mano con una red convolucional entrenada desde cero

Daniel Rodríguez Alonso Carlos Falcón Brito

Aprendizaje Automático 2 · Práctica de redes convolucionales
31 de octubre de 2026

Resumen

Entrenamos desde cero una red convolucional, GestureCNN, que clasifica recortes de la mano de 128×128 px en ocho gestos de HaGRID y una clase nula (*no_gesture*), pensados como órdenes para un robot. Los datos se dividen *por persona*, de modo que las 726 personas de test no aparecen ni en entrenamiento ni en validación. La red tiene cinco bloques convolucionales con normalización por lotes, *dropout* y *global average pooling* (2,36 M parámetros). Tras una búsqueda manual de 9 configuraciones de hiperparámetros, el modelo final se entrena con lotes de 32 y alcanza en test una exactitud del 99,03 % (IC del 95 %: [98,61 %; 99,42 %]), muy por encima del objetivo del 85 %. La búsqueda y un estudio de ablación con dos semillas indican que el resultado es poco sensible a los hiperparámetros y que la normalización por lotes es el único componente con un efecto claro. Un ResNet18 preentrenado (99,20 % en test) no es significativamente mejor, y Grad-CAM muestra que la red se fija en los dedos que distinguen cada gesto.

Palabras clave: reconocimiento de gestos, redes convolucionales, búsqueda de hiperparámetros, HaGRID, ablación, Grad-CAM.

1. Introducción

Los gestos de la mano permiten dar órdenes a un robot sin mando ni contacto, algo útil en entornos ruidosos o cuando el operador tiene las manos ocupadas [1]. Planteamos el problema como la clasificación de un vocabulario cerrado de órdenes más una *clase nula* que deja el robot quieto. De este planteamiento salen dos requisitos. El modelo debe funcionar con personas que nunca vio, así que el rendimiento se mide con personas ausentes del entrenamiento: una partición aleatoria por imagen sería optimista, porque la misma persona, con el mismo fondo, aparecería a ambos lados [2]. Y debe reconocer bien la clase nula, porque confundir una mano en reposo con una orden mueve el robot sin querer.

Objetivos. Diseñar y entrenar desde cero en PyTorch [3] una red convolucional con *pooling*, *dropout* y parada anticipada que, con entre 5 y 15 clases, alcance al menos un 85 % de exactitud en test; documentar una búsqueda manual de hiperparámetros con al menos cinco configuraciones, y analizar el modelo final con sus curvas de pérdida y exactitud y su matriz de confusión. Como ampliaciones, usamos aumento de datos, comparamos con un ResNet18 preentrenado [4] (*transfer learning*) y añadimos un estudio de ablación y mapas Grad-CAM [5].

Enfoque. Como en los sistemas prácticos, separamos detección y clasificación: un detector como MediaPipe Hands [6] localiza la mano y una red clasifica el recorte. Este trabajo se centra en la segunda etapa y usa las cajas anotadas de HaGRID [7], uno de los mayores conjuntos de

gestos de la mano. La red sigue el esquema de bloques de VGG [8], con normalización por lotes (BN) [9], *dropout* [10] y *global average pooling* [11] en lugar de capas densas grandes.

2. Datos

Usamos el subconjunto público `hagrid-sample-30k-384p` [12] de HaGRID, con imágenes de 384 px y las anotaciones originales: la caja de cada mano, su etiqueta y un identificador anónimo de la persona (`user_id`). Elegimos los ocho gestos de la tabla 1, que cubren las órdenes básicas de un robot móvil con pinza. Sus 14.078 imágenes contienen 17.296 manos de 4.877 personas. La clase nula está formada por las 3.219 *segundas manos* de esas mismas imágenes, que HaGRID etiqueta como *no_gesture* porque no hacen el gesto. Así, los negativos comparten persona, fondo e iluminación con los positivos, y la red no puede separar las clases por el contexto. Para equilibrar las clases, submuestreamos la nula hasta 1.760 ejemplos, el tamaño medio de una clase de gesto.

Cada mano se recorta como un cuadrado centrado en su caja, de lado 1,5 veces el lado mayor de la caja (un margen del 25 % por cada lado), y se redimensiona a 128×128 px (figura 1). Las manos son pequeñas (la mediana del lado de la caja es de 68 px), así que la mayoría de los recortes se amplían.

La partición usa `StratifiedGroupKFold` con `user_id` como grupo: 20 trozos estratificados por clase, de los que 14 van a entrenamiento, 3 a validación y 3 a test (70/15/15). Resultan 11.088, 2.375 y 2.374 recortes de 3.416, 735 y 726 personas, sin ninguna persona compartida entre particiones (lo comprueba una prueba automática). La normalización por canal usa estadísticas calculadas solo con entrenamiento.

Tabla 1: Vocabulario de órdenes y recortes por clase y partición, tras equilibrar *no_gesture*. Ninguna persona aparece en más de una partición.

Clase	Orden	Train	Val	Test	Total
stop	Parar	1225	261	262	1748
like	Avanzar	1214	258	260	1732
dislike	Retroceder	1248	268	267	1783
one	Girar a la izquierda	1245	267	266	1778
peace	Girar a la derecha	1237	266	266	1769
fist	Cerrar pinza	1214	259	261	1734
palm	Abrir pinza	1239	267	264	1770
call	Volver al inicio	1234	265	264	1763
no_gesture	Ninguna (quieto)	1232	264	264	1760
Total imágenes		11088	2375	2374	15837
Personas		3416	735	726	4877

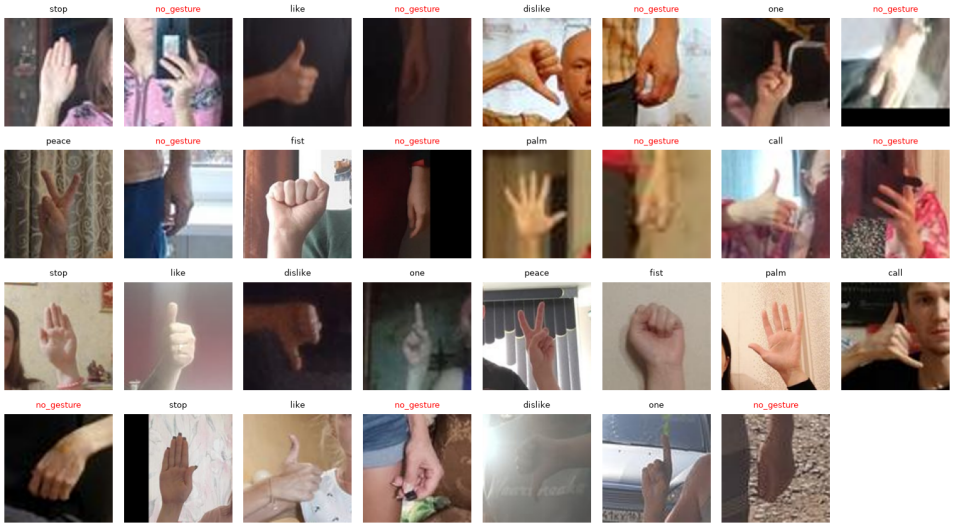


Figura 1: Recortes de 128 × 128 px de las nueve clases. Los de *no_gesture* (en rojo) son manos en reposo, sujetando objetos o parcialmente visibles.

3. Métodos

Aumento de datos. En entrenamiento, cada recorte se rota hasta ±15°, se recorta al azar conservando entre el 80 y el 100 % del área (`RandomResizedCrop` con aspecto 1:1), se voltea horizontalmente con probabilidad 0,5 y se varían el brillo y el contraste (±30 %) y la saturación (±20 %) [13]. Ninguna de estas transformaciones cambia el significado del gesto. Por eso se excluye el volteo vertical, que convierte *like* en *dislike*.

Arquitectura. GestureCNN (figura 2) encadena cinco bloques. Cada uno aplica dos veces la secuencia convolución 3 × 3, BN y ReLU, y termina con un *max pooling* 2 × 2. Los canales crecen de 32 a 256 mientras la resolución baja de 128^2 a 4^2. Los tres últimos bloques añaden `Dropout2d` (0,1, 0,2 y 0,3), que apaga mapas de características completos [14]. La cabeza aplica *global average pooling*, un *dropout* de 0,5 y una capa lineal de 256 a 9 salidas: 2 313 parámetros, frente a los 36 873 de una capa lineal sobre el mapa aplanado. Las variantes de la ablación quitan BN o el *dropout* y dejan intacto el resto de la red.

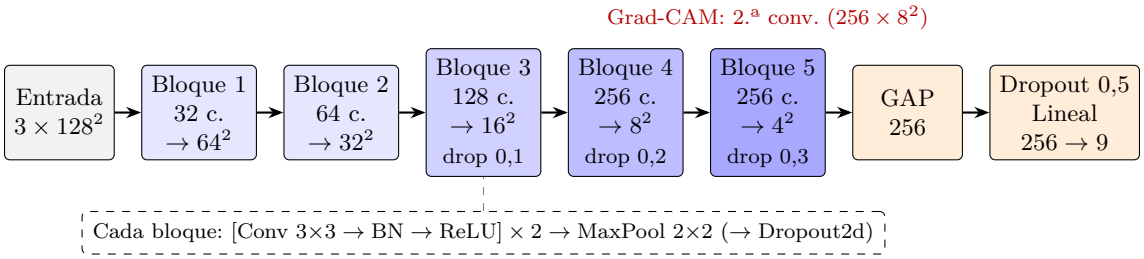


Figura 2: Arquitectura de GestureCNN (2,36 M parámetros): canales, resolución de salida y probabilidad de `Dropout2d` de cada bloque. Grad-CAM se calcula sobre la última convolución.

Entrenamiento. Usamos entropía cruzada sin pesos de clase, porque las clases están equilibradas, y Adam [15] con una tasa de aprendizaje inicial de 10^{-3}, *weight decay* 10^{-4} y lotes de 64 (32 en el modelo final). La tasa se divide por dos tras 3 épocas sin mejorar la pérdida de validación. La parada anticipada [16] detiene el entrenamiento tras 6 épocas sin mejora, con un máximo de 50, y restaura los pesos de la mejor época. La semilla es 42 salvo que se indique otra. Todo se entrenó con PyTorch 2.14.1 en una GTX 1660 Ti de portátil. Como referencia, ResNet18 preentrenado en ImageNet [17] se reajusta entero con el mismo protocolo, una capa final de 9 salidas y una tasa diez veces menor (10^{-4}).

3.1. Búsqueda de hiperparámetros

Partiendo del modelo completo (tasa 10^{-3}, lotes de 64, cinco bloques de 32 a 256 filtros, sin capa oculta y ReLU), hicimos una búsqueda manual en la que cada configuración cambia *un único* hiperparámetro y mantiene fijo todo lo demás, incluidas la semilla y la partición. Probamos 9 configuraciones:

- tasa de aprendizaje de $3 \cdot 10^{-4}$ y de $3 \cdot 10^{-3}$;
- lotes de 32 y de 128;
- la mitad de filtros por bloque (591 k parámetros), cuatro bloques en lugar de cinco (1,18 M) y una capa oculta de 128 neuronas en la cabeza;
- activaciones ELU [18] y LeakyReLU (pendiente 0,1) en lugar de ReLU.

Cada configuración se entrenó una vez y se evaluó solo en validación. El criterio de elección, fijado de antemano, fue la exactitud de validación. Cambiar un único factor cada vez permite atribuirle la diferencia y cuesta una ejecución por valor probado, unos 20 minutos; a cambio, no explora interacciones entre hiperparámetros.

3.2. Protocolo de evaluación

Selección del modelo. Todas las decisiones se tomaron con validación, pero el modelo final se eligió en dos fases. Lo indicamos explícitamente porque afecta a la lectura del test. Primero, el modelo final fue la variante completa, que era la mejor en validación de la ablación. Con ella, con las variantes y con ResNet18 se evaluó el test una vez por modelo. Después hicimos la búsqueda de hiperparámetros, solo con validación. La configuración con lotes de 32 fue la mejor y pasó a ser el modelo final. La decisión se tomó antes de evaluar esa configuración en test, donde también se evaluó una sola vez; pero el test ya se había mirado con el modelo anterior. Por eso presentamos y comparamos los dos (apartado 4.2).

Métricas e incertidumbre. Reportamos la exactitud, la F1 macro y la matriz de confusión normalizada por filas, cuya diagonal es el *recall*. Como las imágenes de una misma persona no

son independientes, los intervalos de confianza del 95 % se calculan por *bootstrap* remuestreando *personas* (2.000 réplicas) [19]. Para comparar dos modelos sobre las mismas imágenes de test usamos la prueba exacta de McNemar [20, 21], y repetimos la ablación con una segunda semilla (43) para medir la variación entre ejecuciones. Estas comparaciones son exploratorias y no se corrigen por comparaciones múltiples. Grad-CAM [5] se calcula sobre la última convolución (8×8) y siempre en validación.

4. Resultados

4.1. Búsqueda de hiperparámetros

La red es poco sensible a los cambios probados (tabla 2 y figura 3): salvo el lote de 32, todas las configuraciones quedan entre el 98,74 % y el 99,07 % de exactitud de validación, a pocas imágenes de la referencia (98,99 %). Para dar una medida del ruido, dos ejecuciones idénticas del modelo completo con distinta semilla obtienen 98,99 % y 98,82 %. Con las tasas de $3 \cdot 10^{-3}$ y $3 \cdot 10^{-4}$ la parada anticipada actúa antes (épocas 29 y 31, frente a 44), sin cambiar la exactitud. Con la mitad de filtros la red rinde igual (99,03 %) con cuatro veces menos parámetros, lo que indica que está sobredimensionada. Quitar el último bloque es el cambio que más empeora el resultado y el que más ralentiza el aprendizaje. La capa oculta no aporta nada, y ELU y LeakyReLU quedan algo por debajo de ReLU.

El único cambio que mejora a la vez la exactitud y la pérdida de validación es el lote de 32 (17 errores, frente a 24): con la misma tasa, da el doble de actualizaciones por época y un gradiente más ruidoso, que actúa como regularizador. Según el criterio fijado, pasa a ser el modelo final. Aun así, su ventaja procede de una sola ejecución y de elegir la mejor de diez configuraciones, así que su exactitud de validación tiende a ser optimista.

4.2. Modelo final

La parada anticipada detuvo el entrenamiento en la época 45 y restauró los pesos de la 39 (figura 4). La exactitud de validación supera el 95 % en la época 6, y las reducciones de la tasa (épocas 21, 36 y 44) producen las mejoras finales. Las curvas de entrenamiento y validación casi se superponen (diferencia final: 0,6 puntos), así que no hay sobreajuste apreciable. Al principio, la exactitud de validación supera a la de entrenamiento porque esta se mide con el *dropout* activo y sobre imágenes aumentadas.

Evaluado una sola vez en test (2.374 imágenes de 726 personas), el modelo final alcanza una exactitud del 99,03 % (IC del 95 %: [98,61 %; 99,42 %]) y una F1 macro del 99,03 %, con 23 errores, unos 14 puntos por encima del objetivo. El modelo completo, el final anterior, obtuvo un 98,99 % (IC: [98,53 %; 99,40 %]) con 24 errores. Sobre las mismas imágenes, 9 las acierta solo el modelo final y 8 solo el completo (McNemar: $p = 1{,}000$). La mejora de validación no se traslada al test, como cabe esperar si se debía en parte al ruido y a la selección: la exactitud de validación del modelo final (99,28 %) supera a la de test, mientras que en el completo coincidían. Ninguna de las conclusiones depende de cuál de los dos se tome como final.

Todas las clases superan el 98 % de F1 (tabla 3); la peor es *stop* (98,1 %). Las confusiones más frecuentes (figura 5) son *palm*→*stop* (6), *stop*→*palm* (3), *dislike*→*no_gesture* (2), *like*→*call* (2). 11 de los 23 errores caen en dos pares de gestos visualmente próximos: *stop*/*palm*, que solo se distinguen por si los dedos están juntos o separados, y *like*/*call*, que se diferencian por el meñique. Para el robot, el error más grave es tomar una mano en reposo por una orden: 5 de los 264 ejemplos de *no_gesture* de test (1,9 %) se clasificaron como algún gesto.

Tabla 2: Búsqueda manual de hiperparámetros, solo en validación (2.375 imágenes). Cada fila cambia un único hiperparámetro de la referencia (tasa 10^{-3}, lote de 64, cinco bloques de 32 a 256 filtros, sin capa oculta, ReLU); todas usan la semilla 42. Pérdida: pérdida de validación en la mejor época. En negrita, la configuración elegida como modelo final.

Hiperparámetro	Valor	Parám.	Épocas (mejor)	Pérdida	Acc.	F1 macro	Errores
Referencia (*completo*)	—	2,36 M	44 (38)	0,0358	98,99 %	98,99 %	24
Tasa de aprendizaje	$3 \cdot 10^{-4}$	2,36 M	31 (25)	0,0329	98,86 %	98,86 %	27
	$3 \cdot 10^{-3}$	2,36 M	29 (23)	0,0357	99,07 %	99,07 %	22
Tamaño de lote	32	**2,36 M**	**45 (39)**	**0,0280**	**99,28 %**	**99,28 %**	**17**
	128	2,36 M	33 (27)	0,0391	98,91 %	98,91 %	26
Filtros	16–128	0,59 M	41 (35)	0,0368	99,03 %	99,03 %	23
Nº de bloques	4	1,18 M	39 (33)	0,0500	98,74 %	98,73 %	30
Capa oculta	128 neuronas	2,39 M	40 (34)	0,0368	98,91 %	98,91 %	26
Activación	ELU	2,36 M	50 (46)	0,0414	98,82 %	98,82 %	28
	LeakyReLU	2,36 M	49 (43)	0,0319	98,78 %	98,78 %	29

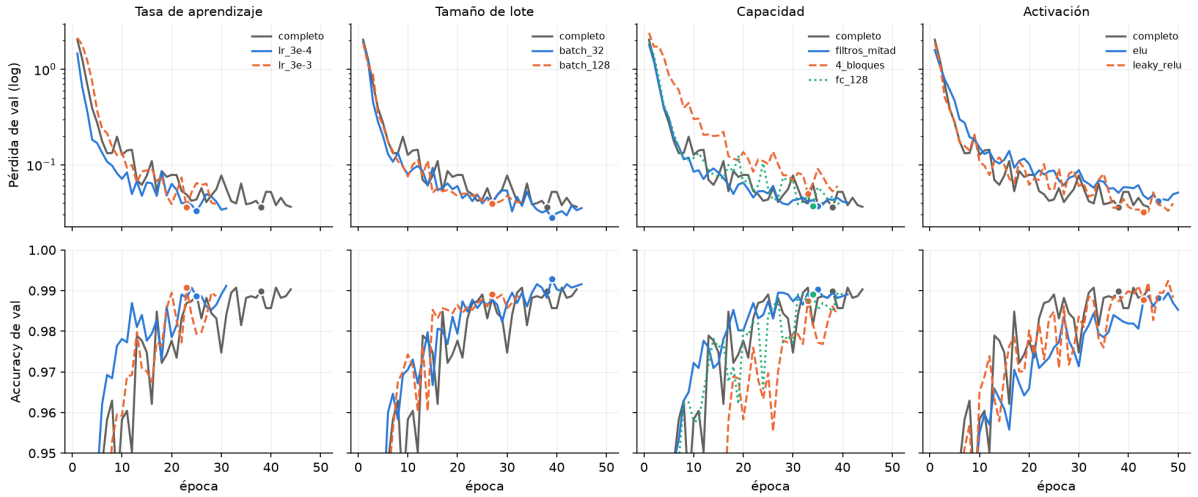


Figura 3: Curvas de validación de la búsqueda, un panel por tipo de hiperparámetro; en gris, la referencia. Arriba, la pérdida (escala logarítmica); abajo, la exactitud por encima del 95 %. El punto marca la mejor época.

4.3. Estudio de ablación

La normalización por lotes es la única ablación con un efecto claro (tabla 4). Sin BN, la exactitud media en test baja al 97,81 %, frente al 99,01 %, y los errores se duplican. De las imágenes en las que solo acierta uno de los dos modelos, 42 las acierta el completo y 11 la variante sin BN ($p < 0{,}001$). Sin BN el aprendizaje es además mucho más lento: tras la primera época la exactitud de validación es del 16,8 %, y no supera el 95 % hasta la época 25 (la 7 en el completo). Con ambas semillas agotó las 50 épocas sin converger, así que el resultado debe leerse como «con el mismo presupuesto de épocas».

Quitar el *dropout* ($p = 0{,}541$) o el aumento de datos ($p = 0{,}701$) no tiene un efecto distinguible de la variación entre ejecuciones: dos ejecuciones idénticas del modelo completo cometen 24 y 23 errores en test. Su huella está en la dinámica del entrenamiento. Sin aumento de datos, la red memoriza el conjunto de entrenamiento, la diferencia entre entrenamiento y validación es la mayor de la ablación (1,3 puntos, frente a 0,5) y la parada actúa antes (épocas 23 y 24).

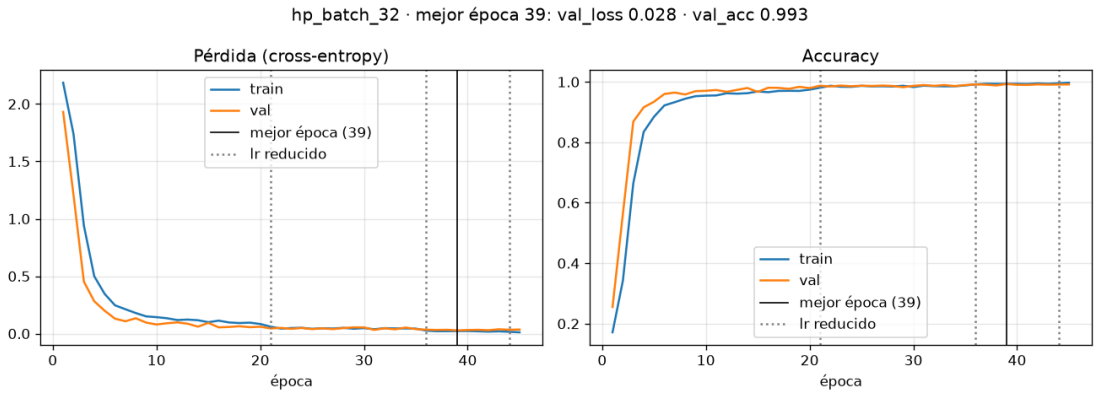


Figura 4: Pérdida y exactitud de entrenamiento y validación del modelo final (lotes de 32). Línea continua: mejor época (pesos restaurados); punteadas: reducciones de la tasa de aprendizaje.

Tabla 3: Métricas por clase del modelo final en test.

Clase	Orden	Precisión	Recall	F1	n
stop	Parar	97,4 %	98,9 %	98,1 %	262
like	Avanzar	99,6 %	98,8 %	99,2 %	260
dislike	Retroceder	99,6 %	98,9 %	99,2 %	267
one	Girar a la izquierda	99,6 %	99,6 %	99,6 %	266
peace	Girar a la derecha	98,5 %	99,2 %	98,9 %	266
fist	Cerrar pinza	100,0 %	100,0 %	100,0 %	261
palm	Abrir pinza	98,9 %	97,7 %	98,3 %	264
call	Volver al inicio	98,9 %	100,0 %	99,4 %	264
no_gesture	Ninguna (quieto)	98,9 %	98,1 %	98,5 %	264

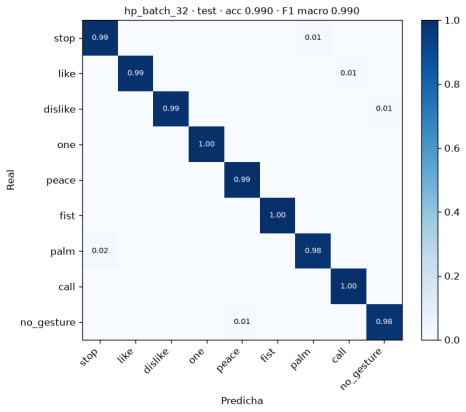


Figura 5: Matriz de confusión del modelo final en test, normalizada por filas (la diagonal es el *recall*).

4.4. Comparación con ResNet18 y coste

ResNet18 preentrenado alcanza un 97,6 % de exactitud de validación tras la primera época (GestureCNN: 25,5 %). En test obtiene un 99,20 % (IC: [98,76 %; 99,57 %]), con 19 errores frente a los 23 de GestureCNN. La diferencia no es significativa: 15 imágenes las acierta solo ResNet18 y 11 solo GestureCNN ($p = 0{,}557$). El preentrenamiento acelera la convergencia, pero no podemos afirmar que un modelo con 4,7 veces más parámetros generalice mejor.

En cuanto al coste, el número de parámetros engaña. GestureCNN tiene 4,7 veces menos parámetros que ResNet18, pero hace 1,6 veces más operaciones por imagen (920 M frente a 592 M MACs), porque sus dos primeros bloques trabajan a 128^2 y 64^2 px y acumulan el 43 % de las operaciones. Aun así, con una sola imagen clasifica un recorte en 8,81 ms en la CPU (Intel Core i7-9750H @ 2.60GHz) y en 1,04 ms en la GPU, frente a 9,70 ms y 2,06 ms de ResNet18. Los dos modelos caben holgadamente en los 33 ms por fotograma de una cámara a 30 fps.

4.5. Explicabilidad con Grad-CAM

En los aciertos de validación, la activación se concentra sobre la mano, no sobre el fondo ni la cara, y señala los rasgos de cada gesto: en *call*, por ejemplo, el mapa tiene dos máximos, sobre el pulgar y sobre el meñique. La figura 6 muestra los errores más seguros, que son de tres tipos:

- **Ruido de etiqueta**: un *no_gesture* que es, de hecho, una palma abierta, y que la red

Tabla 4: Ablación: media de las semillas 42 y 43. Cada variante quita un único componente del modelo completo (lotes de 64); la última columna da las épocas de cada semilla. †ResNet18 se entrenó una sola vez.

				Validación		Test		
Variante	Dropout	BN	Augm.	Acc.	F1 macro	Acc.	F1 macro	Épocas
Completo	sí	sí	sí	98,91 %	98,91 %	99,01 %	99,01 %	44 / 34
Sin dropout	no	sí	sí	99,03 %	99,03 %	98,97 %	98,97 %	29 / 50
Sin BN	sí	no	sí	97,56 %	97,55 %	97,81 %	97,80 %	50 / 50
Sin augmentation	sí	sí	no	98,74 %	98,74 %	98,65 %	98,65 %	23 / 24
ResNet18 preentrenado†	—	sí	sí	99,41 %	99,41 %	99,20 %	99,20 %	23

clasifica como *palm* con probabilidad 1,00.

- **Objetos con forma de gesto**: un teléfono, o el borde vertical de un mueble junto al índice, que la red lee como los dedos de *peace*.
- **Pares próximos o imágenes de baja calidad**: *palm*→*stop* con una separación intermedia entre los dedos, *like* de perfil confundidos con *call* o *one*, y un *palm* pixelado.

En los pares próximos, los mapas de la clase real y de la predicha coinciden: la red mira el lugar correcto, pero la evidencia es ambigua.

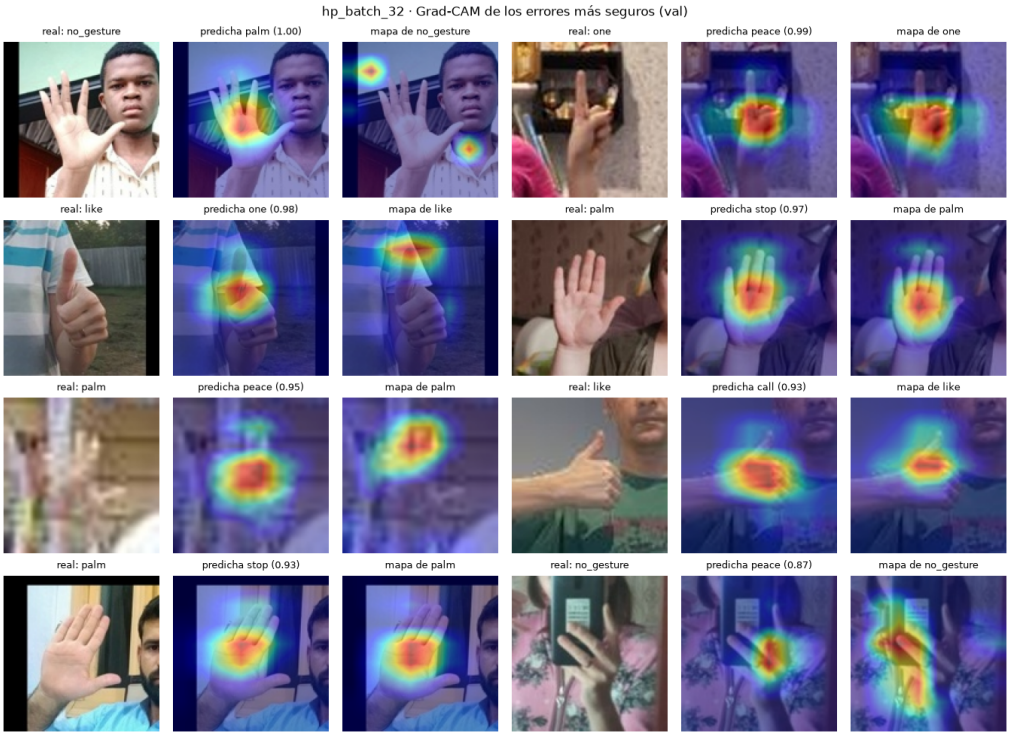


Figura 6: Errores más seguros del modelo final en validación. Para cada uno: la imagen con su clase real, el mapa de la clase predicha y el de la clase real.

5. Discusión

¿Es creíble una exactitud del 99 %? Un resultado tan alto obliga a descartar una fuga de información entre entrenamiento y test [2]. Hemos descartado las más probables: (I) la partición es por persona, sin personas compartidas; (II) las estadísticas de normalización salen solo de entrenamiento; (III) las decisiones se tomaron con validación y el test se evaluó una vez por modelo, con la salvedad del cambio de modelo final explicada en la apartado 3.2; y (IV) validación y test, con personas distintas, dan resultados casi idénticos. Creemos que la exactitud se explica por la propia tarea: los recortes salen de cajas anotadas a mano, así que la mano está siempre centrada y completa, y los gestos de HaGRID están posados para la cámara. Por tanto, la cifra mide el *clasificador* sobre recortes de calidad HaGRID, no un sistema completo con webcam, en el que las cajas de un detector, otra cámara y las posturas de transición entre gestos harán bajar el rendimiento.

Hiperparámetros y componentes. La búsqueda y la ablación coinciden: alrededor de la configuración inicial, el resultado depende poco de la tasa de aprendizaje, del lote, de la capacidad, de la activación, del *dropout* y del aumento de datos, y casi todas las diferencias caen dentro de la variación entre semillas. La excepción es BN, cuyo papel es sobre todo de *optimización*: sin ella, la red no llega a converger en 50 épocas [9]. El *dropout* y el aumento de datos regularizan, pero el modelo completo apenas sobreajusta, porque tiene muchos ejemplos por clase y la parada anticipada ya lo protege. Su valor real debería verse ante el cambio de dominio a la webcam, que nuestro test no representa. El caso del lote de 32, que mejoró la validación pero no el test, ilustra el riesgo de elegir entre muchas configuraciones con un único conjunto de validación.

Limitaciones. Todo se ha medido con las cajas de HaGRID, sin detector ni vídeo propio. A 384 px, las manos pequeñas pierden detalle fino, como la posición del meñique que separa *like* de

call. Usamos solo dos semillas por variante de la ablación y una por configuración de la búsqueda, que además cambia un único hiperparámetro cada vez y no explora interacciones. Por último, algunas etiquetas de *no_gesture* son en realidad gestos, lo que penaliza predicciones correctas.

6. Conclusiones y trabajo futuro

Una red convolucional pequeña, entrenada desde cero, reconoce ocho gestos más una clase nula con una exactitud del 99,03 % (IC del 95 %: [98,61 %; 99,42 %]) en personas que nunca vio, muy por encima del objetivo del 85 %. Las claves del resultado son de diseño más que de capacidad: clasificar recortes de la mano, construir la clase nula con manos de las mismas imágenes y medir con una partición por persona. La búsqueda manual de 9 configuraciones y la ablación muestran que el modelo es robusto a los hiperparámetros y que la normalización por lotes es el único componente imprescindible. Además, con la mitad de filtros la red rinde igual con cuatro veces menos parámetros. Un ResNet18 preentrenado no es significativamente mejor ($p = 0{,}557$), y Grad-CAM confirma que la red decide por la forma de la mano.

Trabajo futuro.

- **Demo en tiempo real con ROS 2.** El diseño previsto encadena captura con OpenCV, detección de la mano con MediaPipe Hands, el mismo recorte de 128×128 usado en el entrenamiento, clasificación con GestureCNN y un filtro de seguridad. Este filtro solo emite una orden si la probabilidad supera 0,7 y la clase es la mayoritaria en los últimos 5–7 fotogramas; si no, o si la clase es *no_gesture*, publica velocidad cero. Las órdenes de la tabla 1 se enviarían como mensajes `Twist` a *turtlesim* en ROS 2 [22], que usa la misma interfaz que Gazebo o un robot real. La latencia medida (8,81 ms en CPU) indica que el clasificador no sería el cuello de botella.
- **Medir el cambio de dominio** con las cajas de MediaPipe y con vídeo propio, y, si es grande, entrenar con recortes del detector.
- **Una red más pequeña**: repetir con varias semillas la configuración con la mitad de filtros y explorar interacciones entre hiperparámetros.
- **Más resolución y una clase nula más rica**: usar HaGRID a 512 px o más, revisar las etiquetas de *no_gesture* y añadir posturas de transición entre gestos.

Reparto del trabajo

El trabajo se repartió por ficheros con contratos fijos entre ambos (`src/common.py`), de modo que cada autor pudiera avanzar con datos o modelos simulados sin esperar al otro.

- **Daniel Rodríguez Alonso**: descarga y verificación de las anotaciones (`download.py`, `show_boxes.py`), recorte de manos (`prepare_data.py`), equilibrado y partición por persona (`make_splits.py`), carga de datos y aumento (`dataset.py`), evaluación y curvas (`evaluate.py`, `plot_curves.py`), Grad-CAM (`gradcam.py`), análisis estadístico y de coste (`analysis.py`), búsqueda de hiperparámetros (`busqueda.py`) y ejecución de los entrenamientos.
- **Carlos Falcón Brito**: propuesta inicial del proyecto, arquitectura GestureCNN y ResNet18 (`model.py`), datos simulados para desarrollar sin el conjunto real (`mock_data.py`), bucle de entrenamiento con parada anticipada (`train.py`), configuraciones de la ablación y lanzador de experimentos (`configs/`, `run_experiments.py`).

Uso de herramientas de IA. Se utilizó un asistente de IA (Claude, de Anthropic) como apoyo en la programación, en la ejecución de los experimentos y en la redacción de este informe. Todas las cifras proceden de los ficheros de resultados del repositorio, generados por los *scripts* descritos. La responsabilidad sobre el contenido corresponde a los autores.

Código. El código, las configuraciones, la partición y los resultados están en `https://github.com/danirodriguezz/gestos-cnn-robot`. Las cifras de este documento se generan automáticamente a partir de `results/` con `src/analysis.py`.

Referencias

[1] Pramod Kumar Pisharady and Martin Saerbeck. Recent methods and databases in vision-based hand gesture recognition: A review. *Computer Vision and Image Understanding*, 141: 152–165, 2015.

[2] Shachar Kaufman, Saharon Rosset, Claudia Perlich, and Ori Stitelman. Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4), 2012.

[3] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.

[4] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

[5] Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-CAM: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 618–626, 2017.

[6] Fan Zhang, Valentin Bazarevsky, Andrey Vakunov, Andrei Tkachenka, George Sung, Chuo-Ling Chang, and Matthias Grundmann. MediaPipe hands: On-device real-time hand tracking. *arXiv preprint arXiv:2006.10214*, 2020.

[7] Alexander Kapitanov, Karina Kvanchiani, Alexander Nagaev, Roman Kraynov, and Andrei Makhliarchuk. HaGRID – HAnd gesture recognition image dataset. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2024.

[8] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *International Conference on Learning Representations (ICLR)*, 2015.

[9] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, volume 37 of *PMLR*, pages 448–456, 2015.

[10] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014.

[11] Min Lin, Qiang Chen, and Shuicheng Yan. Network in network. In *International Conference on Learning Representations (ICLR)*, 2014.

[12] Christian J. Mills. hagrid-sample-30k-384p: subconjunto de HaGRID a 384 px. Hugging Face Datasets, `https://huggingface.co/datasets/cj-mills/hagrid-sample-30k-384p`. Consultado en octubre de 2026.

[13] Connor Shorten and Taghi M. Khoshgoftaar. A survey on image data augmentation for deep learning. *Journal of Big Data*, 6:60, 2019.

[14] Jonathan Tompson, Ross Goroshin, Arjun Jain, Yann LeCun, and Christoph Bregler. Efficient object localization using convolutional networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 648–656, 2015.

[15] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

[16] Lutz Prechelt. Early stopping – but when? In *Neural Networks: Tricks of the Trade*, volume 1524 of *Lecture Notes in Computer Science*, pages 55–69. Springer, 1998.

[17] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. ImageNet: A large-scale hierarchical image database. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009.

[18] Djork-Arné Clevert, Thomas Unterthiner, and Sepp Hochreiter. Fast and accurate deep network learning by exponential linear units (ELUs). In *International Conference on Learning Representations (ICLR)*, 2016.

[19] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall, 1993.

[20] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

[21] Thomas G. Dietterich. Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7):1895–1923, 1998.

[22] Steven Macenski, Tully Foote, Brian Gerkey, Chris Lalancette, and William Woodall. Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66), 2022.